

Chedam — Product & Technical Specification v0.1

Oct 1, 2026 · @sreya brahma teja

1. Overview

Chedam is one offline-first business application that any small business can install on a local hub and run with \$0 in subscriptions, no internet, and no cloud server. A single build is reused across businesses; each business makes it its own through a first-time setup wizard (name, logo, people, storage, settings).

Vision: give small retail, perishables, import/export and service businesses the tools larger companies have — POS, inventory, messaging, expenses, time, HR, reconciliation — on hardware they own, with attractive visuals of margins, profit, loss and stock.

Goals

1. **Zero recurring cost** — no subscriptions, no paid cloud, no per-seat licences.
2. **No internet dependency** — everything works on a local Wi-Fi network; internet is optional for extras only.
3. **Runs 24/7 without failing** — power-loss safe, self-restarting, backed up nightly, recoverable in minutes.
4. **Simple** — one hub, one app, opened in any browser on the shop network.
5. **Reusable across businesses** — branding, people and data are configuration, not code.
6. **Easy switching** — accurate import from the tools a business already uses, with clear reports of what is missing or wrong.
7. **AI-ready** — data and events are structured so offline AI can be added later as a premium tier.

Design principles

- **Offline first:** every device keeps working if the hub or network drops, and catches up after.
- **Every change is an event:** all writes are recorded with who, what, when and which device. This powers sync, audit and future AI.
- **Show, don't list:** numbers appear as charts, tiles and colour-coded status, not raw tables.

- **Ask once, reuse everywhere:** data entered in one module (a vendor, a product) is reused by all others.
- **Flag, never silently fix:** imports, stock conflicts and payroll anomalies are shown to a person to confirm.
- **Light on the hub:** heavy work (imports, analytics) runs in the back-office browser, not on the hub.

2. Scope, non-goals and assumptions

Version 1 is a single-location business on one local network with one hub and roughly 2–10 devices.

In scope (v1)

- First-time setup wizard and multi-business configuration
- Dashboard with attention-required tasks
- POS: till open/close and balances, product catalogue, sales terminal, transactions, returns
- Inventory: stock levels, reserved orders, upcoming orders, damaged and returns
- Messages (user to user) and Pings (device to device), with POS notifications
- Vendors & Clients: records, purchase orders, invoices, communication log
- Expenses: employee out-of-pocket claims with receipt photos
- My Time Here: time sheets, schedules, pay statements
- HR & Payroll: employee records, pay rates, payroll preparation
- Reconciliation: till, bank statement and vendor statement matching
- Import/export of all data (CSV, Excel, JSON)
- Bundled reference datasets (HS codes, nutrition, shelf life, currency)

Not in v1 (non-goals)

- Multiple store locations syncing with each other over the internet
- Integrated card payments (card terminal stays standalone)
- Vendor or customer logins from outside the shop network
- E-commerce storefront
- Automatic government filings (payroll remittances, GST/PST returns)
- AI features (planned as a premium tier; foundations built now)

Assumptions

- The business owns a Wi-Fi router; internet may or may not be present.
- Staff devices have a modern browser (Chrome, Edge, Safari or Firefox from the last 2 years).
- First market is British Columbia, Canada: CAD currency, GST 5% + PST 7% defaults, editable per business.
- English first; text is stored so other languages can be added later.
- Pilot hub is a Raspberry Pi Zero 2 W; production hubs are a Pi 4/5 or a mini PC.

3. Users, roles and permissions

Five built-in roles cover v1; the owner can copy a role and adjust its permissions. Every user also has a personal login (password) and, for till work, a 4–6 digit PIN.

Role	Who	Typical devices
Owner	Business owner; full control, sets up the business	Back-office PC, phone
Manager	Runs daily operations, approves expenses and time sheets	Back-office PC, till, phone
Cashier	Sells, handles returns, opens/closes till	Till
Staff	Stock, receiving, general tasks	Phone, tablet
Accountant (read-mostly)	Reconciliation, reports, exports	Back-office PC

Permissions matrix

V = view, E = create/edit, A = approve, — = no access. "Own" = only their own records.

Area	Owner	Manager	Cashier	Staff	Accountant
Setup & business settings	E	V	—	—	V
Users & roles	E	E (not owners)	—	—	—
Sales terminal	E	E	E	—	—
Refunds / voids	A	A	E (needs approval above limit)	—	V
Till open/close	A	A	E	—	V
Products & prices	E	E	V	V	V

Area	Owner	Manager	Cashier	Staff	Accountant
Stock adjustments, damaged	A	E/A	V	E	V
Purchase orders	A	E	—	V	V
Vendors & Clients	E	E	V	V	V
Expenses	A	A	E (own)	E (own)	V
Time sheets & schedules	A	E/A	E (own)	E (own)	V
Pay statements	E	V	V (own)	V (own)	V
HR records & pay rates	E	V (no pay rates)	—	—	V
Payroll preparation	A	E	—	—	E
Reconciliation	A	E	—	—	E
Import / export	E	E	—	—	E (export)
Messages	E	E	E	E	E
Pings (device)	E	E	E	E	—
Audit log	V	V	—	—	V

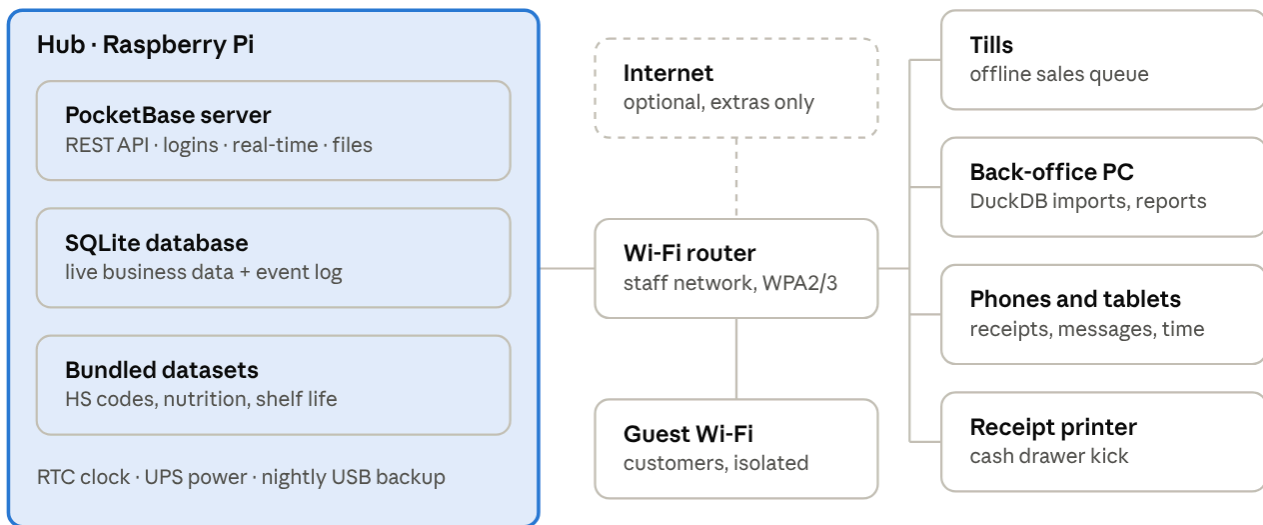
Devices as identities

Each device (till, back-office PC, phone) is registered once with a name ("Till 1", "Back office") and a device key. Pings are addressed to devices, not people, so a till shows a notification no matter who is logged in. A manager can revoke a lost or stolen device from the hub.

4. System architecture

One hub device on the shop's Wi-Fi network holds all shared data and serves the app; every till, PC and phone connects to it over the local network. The internet is never required.

One hub on the shop Wi-Fi serves every device, no internet needed



system architecture · hub, router, devices

Devices talk only to the hub (never directly to each other); a ping from Till 1 to Till 2 goes through the hub in real time. The internet link is dashed because it is optional.

How data flows

1. A device opens `https://chedam.local`; the hub serves the app, which installs itself for offline use.
2. The user logs in; the hub checks credentials and role and returns a session.
3. The device downloads the data it needs (catalogue, prices, tasks) and subscribes to real-time updates.
4. Every change (a sale, a message, a stock count) is sent to the hub, checked against permissions and business rules, saved to SQLite, and written to the event log.
5. The hub pushes the change to every subscribed device within a second; tills update stock, the dashboard updates tiles, pings pop up on POS screens.
6. If a till loses the hub, it keeps working from its local cache and queues changes; on reconnect it uploads the queue and receives what it missed.
7. Heavy jobs (imports, analytics) run in the back-office browser with DuckDB-WASM and send only clean results to the hub.

Why a hub instead of a shared drive

- Real-time messages and pings without polling.
- Phones and tablets work in a browser (browsers cannot open network drives).
- One place enforces permissions and business rules.

- No file-locking corruption from many devices writing at once.

5. Hardware components

The pilot runs on the Raspberry Pi Zero 2 W already owned; production moves the same software to a Pi 4/5 or mini PC by copying it over.

Hub: pilot vs production

	Pilot: Pi Zero 2 W	Production: Pi 4/5 (2–4 GB) or mini PC
CPU / RAM	4-core 1 GHz / 512 MB	4-core 1.5–2.4 GHz / 2–8 GB
Storage	High-endurance microSD 32–64 GB	USB SSD 128–256 GB (boot + data)
Network	2.4 GHz Wi-Fi; Ethernet via USB adapter	Built-in Gigabit Ethernet
Clock	Add DS3231 RTC module	Pi 5 has RTC (battery needed); Pi 4 needs DS3231
Devices served	~2–10, light traffic	10–30+
Heavy work (imports, analytics)	Never on hub — back-office browser	Can run on hub if needed
Role	Development, testing, single-till pilot	Customer installations

Pi Zero 2 W limits to design around

- **512 MB RAM:** hub runs only the app server and SQLite (~30–80 MB). No DuckDB, no builds, no AI on the hub.
- **microSD wear:** use a high-endurance card, SQLite WAL mode, minimal logging to card, and nightly backups to USB.
- **No real-time clock:** without internet the time resets on reboot. The RTC module is mandatory, not optional.
- **2.4 GHz Wi-Fi only, no Ethernet port:** a USB-Ethernet adapter on an OTG hub gives a stable wired link to the router.
- **Single micro-USB data port:** needs a small powered OTG hub for Ethernet + backup drive.

Network

- **Router:** any modern Wi-Fi router with WPA2/WPA3. Two networks: *Staff* (devices + hub) and *Guest* (customers, isolated).
- **DHCP reservation:** the hub always gets the same IP (e.g. 192.168.1.10).
- **Placement:** hub wired to the router; router and hub on the same UPS.

Power

- Official Raspberry Pi power supply (Zero 2 W: 5 V 2.5 A micro-USB; Pi 5: 5 V 5 A USB-C).
- Small UPS covering hub + router (a 600 VA unit runs them for hours), or a Pi UPS HAT for the pilot.
- Goal: power blips never reach the hub; long outages trigger a clean shutdown.

POS peripherals

Device	Type	Connects to	Notes
Barcode scanner	USB HID (acts as keyboard)	Till	No driver needed; phones can use the camera instead
Receipt printer	80 mm thermal, ESC/POS, network (Ethernet/Wi-Fi)	Hub prints to it	Network type avoids browser print dialogs
Cash drawer	RJ11/RJ12 printer-driven	Receipt printer	Printer sends the "open drawer" pulse
Label printer (optional)	Thermal label, network or USB	Hub or back office	Price and expiry labels for perishables
Card terminal	Standalone from bank/processor	Not connected in v1	Amount keyed manually; integration later
Customer display (optional)	Second screen or tablet	Till browser	Shows cart total

Pilot shopping list (beyond the Pi Zero 2 W)

Item	Approx. CAD
High-endurance microSD 64 GB	15–20

Item	Approx. CAD
Official power supply	12–15
DS3231 RTC module	5–10
Powered USB OTG hub + USB-Ethernet adapter	15–25
USB backup drive 32–64 GB	10–15
Case with heatsink	10–15
Small UPS or Pi UPS HAT	30–70
Total	~100–170

Prices are approximate and vary by store; a spare pre-loaded microSD card (~15) is recommended as a 5-minute recovery option.

6. Hub software stack

The hub runs one small server program (PocketBase) on Raspberry Pi OS Lite, plus a few system services that keep it alive, on time and backed up. All components are free and open source.

Layer	Component	Purpose	Free / licence
Operating system	Raspberry Pi OS Lite 64-bit (Debian based)	No desktop; minimal RAM use	Free
App server	PocketBase (single Go binary)	REST API, logins, real-time events, file storage, admin UI, server-side hooks (JavaScript)	MIT
Database	SQLite (embedded in PocketBase), WAL mode	All live business data	Public domain
Web server / HTTPS	PocketBase built-in, or Caddy in front	Serves the app and API over HTTPS on the LAN	Apache 2.0
Local certificates	mkcert or Caddy internal CA	Trusted HTTPS without internet	MIT / Apache 2.0
Name discovery	Avahi (mDNS)	Devices reach the hub at <code>chedam.local</code>	LGPL

Layer	Component	Purpose	Free / licence
Time	DS3231 RTC + chrony	Correct time without internet; hub serves time to devices	GPL
Process supervision	systemd (Restart=always)	Restarts the app if it crashes	Built in
Freeze protection	Hardware watchdog (bcm2835_wdt)	Reboots the Pi if the system hangs	Built in
Backups	Cron job: SQLite online backup + file copy to USB	Nightly, 14-day rotation, checksum verified	Built in
Print service	Small hook/script sending ESC/POS to the network printer	Receipts and drawer kick	Free
Health monitor	App page + small script	Disk space, temperature, last backup, device count, clock status	Our code
Updates	Signed update bundle on USB	Install new versions offline	Our code

Why PocketBase

- One file to install and run; uses ~30–80 MB RAM, fits the Pi Zero 2 W.
- Gives logins, roles, a database, real-time subscriptions (for Messages and Pings), file uploads (for receipts and logos) and an admin dashboard out of the box.
- Business rules (e.g. "refund over \$50 needs manager approval") run as server-side hooks.
- If we outgrow it, data stays in standard SQLite and can move to another server.

Hub file layout

```

/opt/chedam/
  pocketbase          # server binary
  pb_public/          # the built client app (PWA)
  pb_hooks/           # business rules, print service, task engine
  pb_migrations/      # database schema versions
  pb_data/            # live database + uploaded files
  datasets/           # bundled reference data (read-only)
  config/             # certificates, hub settings
/mnt/backup/          # USB backup drive

```

Network services on the hub

Port	Service	Who connects
443	App + API (HTTPS)	All staff devices
123	Time (NTP via chrony)	All staff devices (optional)
22	SSH (key only, disabled by default)	Support / owner
9100 (outbound)	Receipt printer	Hub to printer

7. Client application

The client is one installable web app (PWA) served by the hub; tills, PCs, tablets and phones open `https://chedam.local` and install it to the home screen or desktop. No app store, no per-device installer.

Technology

Concern	Choice	Why
UI framework	Svelte (SvelteKit, static build)	Small, fast bundles on old tills and phones
Styling	Tailwind CSS + our design tokens	Consistent look; brand colours swapped per business
Charts	Apache ECharts	Margins, profit/loss, stock trends; works offline
Tables	Lightweight data-grid component	Sort, filter, inline edit

Concern	Choice	Why
Local storage	IndexedDB (via Dexie.js)	Offline cache and outgoing sales queue
Offline shell	Service worker	App opens with no hub connection
Real-time	PocketBase SDK (Server-Sent Events)	Messages, pings, live stock
Import/analytics	DuckDB-WASM + SheetJS + Papa Parse	Runs in the back-office browser
Barcode (camera)	Browser BarcodeDetector API, ZXing fallback	Phone scanning
Build tools	Node.js + Vite on the developer PC	Never build on the hub

Layout

- **Top bar:** business logo and name, logged-in user, device name, connection status (green = live, amber = offline, queued items count), notifications bell.
- **Left menu:** Dashboard, POS, Inventory, Messages, Pings, Vendors & Clients, Expenses, My Time Here, HR & Payroll, Reconciliation, Reports, Settings. Items hidden by role.
- **Main area:** module screens. POS screens are touch-first with large buttons.
- **Responsive:** phone layout collapses the menu into a bottom bar.

Offline behaviour

- Every device caches the product catalogue, prices, tax rules and its user list.
- If the hub is unreachable, the till keeps selling. Sales, returns and till actions are saved to the local queue with a unique ID and device timestamp.
- On reconnect, the queue uploads in order. The hub accepts each item once (idempotent IDs), so nothing is duplicated.
- Conflicts (e.g. stock went below zero) are accepted and raised as tasks on the dashboard, never blocking a sale.
- Modules that need fresh shared data (HR, payroll, reconciliation) show read-only while offline.

Notifications

- While the app is open, messages and pings arrive instantly via the live connection, with a toast, a sound and a badge.

- On a POS, a ping shows as a banner over the sales screen that the cashier acknowledges with one tap.
- **Limitation:** true push notifications to closed apps need an internet push service, so v1 notifies only while the app is open. Tills stay open all day, so this suits POS use.

Printing and scanning

- Receipts: the till sends the sale to the hub; the hub formats ESC/POS and sends it to the network printer, which also opens the cash drawer.
- Reports and invoices: generated as PDF in the browser for normal printing or saving.
- Scanning: USB scanners type the code into the focused field; phones use the camera.

8. Data layer and core data model

All live data sits in one SQLite database on the hub; DuckDB runs in the back-office browser for imports and analytics and reads exported snapshots. Money is stored as integer cents; quantities as decimals with a unit; all times in UTC with the business time zone in settings.

Storage roles

Store	Where	Holds
SQLite (live)	Hub	All business records, users, events
File storage	Hub <code>pb_data/storage</code>	Logos, receipt photos, attachments
Reference datasets	Hub <code>datasets/</code> (read-only SQLite)	HS codes, nutrition, shelf life, currency history
IndexedDB	Each device	Cached catalogue, offline queue
DuckDB-WASM	Back-office browser (temporary)	Import staging, profiling, analytics
Backups	USB drive	Nightly database + files

Standard fields on every table

`id` (unique, created on the device so offline records never collide), `created_at`, `updated_at`, `created_by` (user), `device_id`, `deleted_at` (soft delete — nothing is truly erased except by retention rules).

Core entities

Group	Table	Key fields
Business	<code>business</code>	name, legal name, logo, brand colours, address, tax numbers (GST/BN), currency, time zone, fiscal year start
	<code>location</code>	name, address (one in v1; ready for more)
	<code>settings</code>	key/value: tax rates, refund limits, till float, receipt footer
People	<code>users</code>	name, email, phone, role, PIN hash, status, photo
	<code>roles</code> / <code>permissions</code>	role name, permission list
	<code>employees</code>	user link, start date, job title, pay type (hourly/salary), pay rate, emergency contact, SIN (encrypted, owner-only)
	<code>devices</code>	name, type (till/PC/phone), key, last seen, assigned printer
Catalogue	<code>products</code>	SKU, barcode(s), name, category, unit, cost, price, tax class, HS code, origin country, perishable flag, shelf-life days, storage temp, allergens, nutrition link, reorder point, image
	<code>categories</code>	name, parent, margin target
	<code>price_history</code>	product, old/new price and cost, date, by
Stock	<code>stock_levels</code>	product, location, on hand, reserved, incoming
	<code>stock_lots</code>	product, lot/batch, received date, expiry date, quantity, cost
	<code>stock_movements</code>	product, lot, type (sale, receive, adjust, damage, return, transfer), qty, reason, reference
Sales	<code>tills</code>	device, float, opened by/at, closed by/at, expected vs counted cash, difference
	<code>sales</code>	number, till, cashier, customer, subtotal, tax, discount, total, status

Group	Table	Key fields
	<code>sale_lines</code>	sale, product, lot, qty, unit price, discount, tax, cost at sale (for margin)
	<code>payments</code>	sale, method (cash, card, other), amount, reference
	<code>returns</code>	original sale, lines, reason, refund method, approved by
Parties	<code>parties</code>	type (vendor, client, both), name, contacts, addresses, payment terms, currency, tax ID, notes
	<code>purchase_orders</code> / <code>po_lines</code>	vendor, status (draft, sent, partial, received, closed), expected date, lines
	<code>customer_orders</code> / <code>order_lines</code>	client, status, reserved stock, due date
	<code>invoices</code> / <code>invoice_lines</code>	party, direction (payable/receivable), due date, status, amounts, attachment
Comms	<code>conversations</code> / <code>messages</code>	participants, text, attachments, read receipts
	<code>pings</code>	from device/user, to device, text, priority, acknowledged by/at
Money	<code>expenses</code>	employee, date, vendor, amount, tax, category, receipt photo, status, approved by, reimbursed on
	<code>bank_accounts</code> / <code>bank_lines</code>	imported statement lines, matched record
	<code>reconciliations</code>	type (till, bank, vendor), period, status, differences
Time & pay	<code>shifts</code> (schedule)	employee, start, end, role
	<code>time_entries</code>	employee, clock in/out, breaks, device, approved
	<code>pay_runs</code> / <code>pay_stubs</code>	period, gross, deductions, net, status
System	<code>tasks</code>	type, title, link to record, assigned role/user, priority, due, status

Group	Table	Key fields
	<code>events</code> (audit/change log)	who, device, table, record, action, before/after, time
	<code>import_jobs</code>	source file, mapping template, counts, issues, status

The event log

Every create, update and delete writes an `events` row with the before/after values. This single table gives: the audit trail (who changed a price, who voided a sale), offline sync replay, undo for mistakes, history charts, and the training/context data that future AI features will read.

Schema changes

The schema is versioned with PocketBase migrations. Each app update carries its migrations; the hub backs up before applying them and rolls back on failure.

9. First-time setup and multi-business configuration

The same Chedam build ships to every business; on first start the hub runs a setup wizard that turns it into that business's system. Nothing business-specific lives in code.

How one app serves many businesses

- **One hub = one business.** Each business has its own hub, database and files; no data is ever shared between businesses.
- **Branding is data:** logo, colours, receipt header/footer and business name come from the `business` and `settings` tables.
- **Business type presets:** the wizard offers a starting template (grocery/perishables, general retail, food service, import/export, services) that pre-fills categories, tax classes, reorder rules and dashboard widgets. Everything stays editable.
- **Feature switches:** modules a business does not need (e.g. Payroll) can be hidden.

Wizard steps

1. **Welcome and language** — choose language, accept local licence terms, set hub date/time (checked against the RTC).
2. **Business profile** — trading name, legal name, business type, address, phone, email, GST/HST number, PST number, Business Number, currency, time zone, fiscal year start.

3. **Branding** — upload logo; pick or auto-extract brand colours from the logo; preview a receipt and dashboard.
4. **Owner profile** — name, email, phone, password, PIN, recovery code (printed/saved; the only way back in if the password is lost with no internet).
5. **Managers and staff** — add people now or later: name, role, contact, PIN; optional pay type and rate for HR.
6. **Locations and storage** — store name and address; storage areas (shelf, cooler 0–4 °C, freezer –18 °C, dry store, warehouse) with temperature ranges used by perishable alerts.
7. **Devices** — name this device; generate a pairing code/QR for each till, PC and phone to join.
8. **Peripherals** — find the receipt printer on the network, print a test page, test drawer kick.
9. **Taxes and payments** — confirm GST/PST rates and tax classes; payment methods accepted (cash, debit, credit, gift card, store credit); till float amount.
10. **External IDs and connections (optional)** — bank name and account nickname for reconciliation, card processor merchant ID, CRA payroll account number, customs/importer number (CBSA BN-RM), accountant contact. Stored as references; live connections need internet and stay off by default.
11. **Bring your data** — import products, stock, customers, vendors from a file (Section 12), or start with the preset sample catalogue, or start empty.
12. **Backups** — insert USB drive, choose backup time (default 02:00), run first backup.
13. **Review and finish** — summary, then the dashboard opens with a "Getting started" task list.

Wizard rules

- Each step saves as it goes; the wizard can be resumed.
- Required: business name, owner account, tax settings, one device. Everything else can be skipped and appears later as a dashboard task.
- The wizard can be re-run from Settings for edits, never to wipe data (wiping requires owner password + recovery code).

10. Dashboard and attention engine

After login the dashboard answers one question: *what needs me now?* An attention list sits on top, KPI tiles and charts below, both filtered by the user's role.

Attention list

A rules engine on the hub creates **tasks** automatically; users can also create tasks manually. Each task has a priority (Urgent, Today, This week), owner (role or person), a link straight to the record, and closes itself when the condition is fixed.

Rule	Trigger	Goes to	Priority
Low stock	On hand $\leq$ reorder point	Manager	Today
Out of stock	On hand $\leq$ 0, or negative after offline sync	Manager	Urgent
Expiring soon	Lot expiry within N days (per category, default 3)	Staff, Manager	Today
Expired	Lot past expiry with stock remaining	Manager	Urgent
Wrong storage	Product storage temp not matching its storage area	Staff	This week
Till variance	Counted vs expected cash differs > threshold (default \$5)	Manager	Today
Till not closed	Till open past closing time	Manager	Urgent
Refund needs approval	Refund over limit	Manager	Urgent
PO overdue	Expected date passed, not received	Manager	Today
Invoice due	Payable due within 7 days / overdue	Owner, Accountant	Today
Receivable overdue	Client invoice past due	Owner	This week
Expense pending	Claim waiting for approval	Manager	This week
Time sheet pending	Period ended, not approved	Manager	Today
Missed punch	Clock-in without clock-out after shift end + 2 h	Manager	Today
Payroll due	Pay date in 3 days and run not prepared	Owner	Urgent
Reconciliation open	Bank/till period not reconciled by day X	Accountant	This week

Rule	Trigger	Goes to	Priority
Margin drop	Product sold below cost or below category margin target	Owner	Today
Backup failed	Nightly backup missing or failed	Owner	Urgent
Hub health	Disk > 80%, temp high, clock unsynced	Owner	Urgent
Import issues	Import finished with unresolved issues	Importer	Today
Setup incomplete	Wizard steps skipped	Owner	This week

KPI tiles and charts (owner/manager view)

- **Today:** sales, transactions, average basket, gross margin %, cash in tills.
- **Sales trend:** line chart, last 30 days vs previous 30.
- **Margin by category:** bars, coloured against target (green above, amber near, red below).
- **Profit & loss (month to date):** revenue, cost of goods, expenses, payroll, net — as a waterfall.
- **Inventory health:** stock value, slow movers, items expiring this week, stock-outs.
- **Wastage:** value of damaged/expired stock this month.
- **Payables/receivables:** due this week, overdue.

Cashier/staff view

My tasks, my schedule this week, unread messages, pings for this device, quick buttons (Start sale, Clock in/out, Submit expense).

11. Module specifications

Each module below lists its screens, key behaviours and rules. All modules share the same records (products, parties, people), write to the event log, and feed the attention engine.

11.1 POS

Till balances — open and close

- Open: select till, enter float by denomination (or accept default), cashier PIN.

- During shift: cash drops and pay-outs (e.g. paid a delivery driver) recorded with reason.
- Close: count cash by denomination; app shows expected cash (float + cash sales – cash refunds – drops – pay-outs), counted, and difference. Card totals entered from the card terminal's end-of-day slip.
- Close produces a Z-report (sales by method, tax, refunds, voids, discounts) printed and stored.
- Variance over threshold needs a manager note and raises a task.

Product catalogue

- List with search, barcode scan, category filter, stock and margin columns.
- Product form: name, barcodes, category, unit (each, kg, lb, case), cost, price, live margin % and markup %, tax class, perishable details (shelf life, storage temp, allergens), HS code and origin for imports, reorder point, image.
- Autofill from bundled datasets: scanning a known barcode pre-fills name, brand, ingredients, allergens, nutrition; typing a description suggests HS codes and shelf life.
- Variants (size/weight) and case-to-unit conversion (1 case = 12 units).
- Bulk edit prices; price changes saved to price history.

Sales terminal

- Touch-first: product grid by category, search, scan, weighed items (enter weight or read scale later).
- Cart: quantity, line discount, cart discount (permission-limited), customer attach, notes.
- Payment: cash (with change calculation), card (amount sent to standalone terminal, cashier confirms approved), split payments, store credit.
- Receipt: print, email later (when internet exists), or none.
- Hold/recall a sale; void a line; void a sale (manager PIN).
- Perishables: sells from the oldest lot first (FEFO — first expiry, first out); expired lots blocked unless manager overrides.
- Works fully offline (Section 7).

Transactions

- Searchable list of sales, returns, voids by date, till, cashier, customer, amount, product.
- Detail view, reprint receipt, start a return from here.

Returns

- Find original sale (scan receipt barcode or search); select lines and quantities.

- Reason (damaged, wrong item, quality, expired, other) and restock decision: back to stock or to damaged.
- Refund to original method or store credit; above-limit refunds need manager approval.
- Returns without receipt allowed only with manager approval.

11.2 Inventory

Stock levels

- Per product: on hand, reserved, available (on hand – reserved), incoming, by storage area and by lot with expiry.
- Colour status: green OK, amber low, red out, purple expiring.
- Stock count (full or cycle count): print/scan sheet, enter counts, review differences, approve adjustments.
- Stock value at cost and at retail; slow movers (no sale in N days).

Orders — reserved

- Client orders that hold stock (e.g. a restaurant's weekly order). Reserving lowers available stock; fulfilling turns it into a sale or invoice.

Upcoming (incoming)

- Purchase orders from vendors: draft, send (print/PDF/email later), partial and full receiving.
- Receiving screen: scan items, enter actual qty, lot number, expiry, cost; differences flagged against the PO.
- Import shipments: HS code, origin, customs reference, landed cost (freight, duty, brokerage) spread across lines so margins reflect true cost.
- Reorder suggestions: based on sales velocity (7-day and 28-day average), lead time and reorder point.

Damaged & returns

- Record damaged, spoiled, expired or customer-returned stock with reason, photo and lot.
- Return-to-vendor: create vendor return/credit request linked to the original PO.
- Wastage report by product, reason and value.

11.3 Messages (person to person)

- One-to-one and group conversations between logged-in users; an "Everyone" channel for announcements.

- Text, emoji reactions, photo/file attachments, mentions, read receipts, reply-to-message.
- Messages follow the person: they see them on whichever device they log into.
- Link a record into a message (a product, PO, task) as a clickable chip.
- Manager announcements can be pinned and require "acknowledged".
- Retention setting (default: keep 1 year).

11.4 Pings (device to device)

- Send to one device, several devices or all tills: "Till 2 needs change", "Customer pickup at front".
- Pings follow the device, not the person; show as a banner on POS screens with sound, acknowledged with one tap.
- Reply in place: the receiving device can reply, and the reply returns to the sending device.
- Quick templates (configurable): Need change, Need help, Price check, Manager to till, Delivery arrived.
- Priority: normal (banner) or urgent (full-screen with repeat sound until acknowledged).
- Unacknowledged pings after N minutes escalate to managers.

11.5 Vendors & Clients

Every third party is a **party** with type vendor, client or both.

- Profile: name, contacts (name, role, phone, email), addresses, payment terms (e.g. Net 30), currency, tax ID, importer/exporter details, notes, documents.
- Vendor side: products they supply with vendor SKU and cost, purchase orders, receiving history, bills (payables), payments made, returns/credits, performance (on-time %, short-shipped %, damage %).
- Client side: customer orders, invoices (receivables), payments received, statements, credit limit, purchase history.
- Communication log: notes, calls, messages and attached documents per party, with follow-up reminders.
- Multi-currency: amounts in party currency with exchange rate captured on the document date (from bundled/cached rates or entered manually).
- v1 parties do not log in; a party portal is a later, internet-dependent feature.

11.6 Expenses (employee out-of-pocket)

- Employee submits: date, vendor/store, amount, tax amount, category, payment (personal card/cash), note, receipt photo (phone camera).
- Flow: Submitted → Approved/Rejected (manager, with reason) → Reimbursed (cash from till, cheque, e-transfer or added to next pay).
- Duplicate check: same amount + date + vendor flagged.
- Missing receipt above a limit is flagged.
- Feeds P&L by category and the reconciliation module.

11.7 My Time Here

Time sheets

- Clock in/out on any device with PIN (optional photo); breaks.
- Weekly view per employee; edits by manager are logged with reason.
- Manager approves per pay period; anomalies flagged (missed punches, overtime, off-schedule hours).

Schedules

- Weekly roster grid by person and role; drag to create shifts; copy last week.
- Publish schedule; staff see their own on their device; shift swap requests.
- Labour cost per day vs sales shown to managers.

Cheques / statements

- Employees view their pay stubs (gross, deductions, net, year to date) and download PDFs.
- Record of how they were paid (cheque number, direct deposit reference).
- T4 summary data prepared at year end (filing itself is outside v1).

11.8 HR & Payroll

- Employee records: personal details, emergency contact, job title, start date, pay type and rate, vacation entitlement (BC default 4% / 2 weeks), documents (contract, TD1 forms), SIN encrypted and owner-only.
- Leave: vacation, sick, unpaid; balances accrue automatically.
- Payroll preparation: pulls approved hours, overtime (BC: over 8 h/day or 40 h/week at 1.5×), vacation pay, reimbursements; calculates gross.
- Deductions (CPP, EI, income tax): v1 lets the owner enter them from the CRA PDOC calculator or the accountant; automatic calculation is a later phase because rates change every year and must be verified.

- Pay run review screen: line by line, anomalies flagged, owner approves; pay stubs generated.
- Outputs: pay stubs, payroll journal for accounting, remittance summary for the owner/accountant.

11.9 Reconciliation

- **Till reconciliation:** each closed till matched to its sales and card slip; variances explained.
- **Bank reconciliation:** import bank statement (CSV/OFX/QFX from online banking); auto-match lines to sales deposits, vendor payments, expenses and payroll by amount, date and reference; manual match for the rest; unmatched items listed.
- **Vendor statement reconciliation:** compare a vendor's statement with our bills and payments; list differences.
- **Period close:** mark a month reconciled; locked periods need owner unlock to edit.
- **Outputs:** reconciliation report and export for the accountant (CSV/Excel, and QuickBooks/Xero-compatible formats).

12. Data import and export (DuckDB)

Import is how a business switches to Chedam, so it must be accurate, explain itself and never write bad data silently. It runs in the back-office browser with DuckDB-WASM, so the hub only receives clean, validated records.

Supported sources

- **Files:** CSV, TSV, Excel (.xlsx/.xls), JSON, Parquet; zip bundles of several files.
- **Known exports (saved templates):** Square, Shopify, Lightspeed, Clover, Loyverse, QuickBooks (CSV/IIF), Xero, Wave, Zoho Inventory, generic spreadsheets.
- **Bank statements:** CSV, OFX, QFX.

Import pipeline

1. **Upload:** drop one or many files; each becomes an `import_job`.
2. **Detect:** file type, encoding, delimiter, header row, sheets, date and number formats (e.g. 01/02/2026, 1.234,50).
3. **Profile (DuckDB):** for each column — type, empty %, distinct values, min/max, sample values, pattern (barcode, email, phone, money, date).
4. **Identify the table:** score each file/sheet against Chedam entities (products, stock, customers, vendors, sales, invoices, employees) from its columns and values.

5. **Map columns:** match headers using a synonym dictionary ("Item Name", "Product", "Description" → `products.name`), fuzzy matching and value patterns. Each mapping gets a confidence: high (auto), medium (suggested, confirm), low (user must choose).
6. **Transform:** unit and currency normalisation, split/merge columns ("Full name" → first/last), tax-inclusive to tax-exclusive prices, date parsing, trimming, case fixes.
7. **Validate:** required fields, types, ranges (price ≥ 0), duplicates (same barcode twice), referential integrity (sale lines point to known products), cross-checks (cost > price warns about negative margin).
8. **Gap report:** which required entities are missing entirely (e.g. "no vendor list found — 300 products reference 12 unknown vendors") and which records are incomplete.
9. **Preview and fix:** a grid of issues grouped by type; fix in place, bulk-fix, exclude rows, or create missing records (e.g. auto-create the 12 vendors).
10. **Enrich (optional):** fill blanks from bundled datasets (nutrition, allergens, HS codes) — marked as suggested.
11. **Commit:** sent to the hub in one transaction; all or nothing; the import job stores counts and can be rolled back.
12. **Save template:** the mapping is saved ("Square items export") so the next import is one click.

Issue types reported

Issue	Example	Default action
Missing required field	Product without price	Block row until fixed
Wrong type	"N/A" in quantity	Block row
Duplicate	Two products with barcode 0123...	Ask: merge, keep both, skip
Orphan reference	Sale for unknown SKU	Offer to create or map
Missing table	Products reference vendors, no vendor file	Gap report + create stubs
Suspicious value	Price \$0, cost > price, date in future	Warn, allow
Unit mismatch	Mixed kg and lb	Ask for conversion

Export

- Any list screen exports to CSV, Excel or PDF.

- **Full export:** entire business as a zip of CSV + JSON with a data dictionary — the business can always leave with its data.
- **Accountant exports:** sales journal, expenses, payroll journal, reconciliation in QuickBooks/Xero-compatible CSV.
- **Backup format:** the raw SQLite database + files, restorable on another hub.

13. Bundled reference datasets

Reference data ships inside Chedam as compact read-only SQLite files, so users never download or configure anything. A dataset build script (run by us, not the user) downloads sources, trims them, and packs them with version and licence notes.

Dataset	Source	Used for	Licence / condition	Target size
HS codes (6-digit)	UN Comtrade classification files (HS 2022)	HS suggestions, import/export docs	Free; WCO holds rights to official text	< 5 MB
Canada tariff (8-10 digit)	CBSA Customs Tariff	Duty rates, import landed cost	Open Government Licence - Canada	< 20 MB
India ITC-HS	DGFT schedules	Export codes for India-origin goods	Public; needs parsing	< 10 MB
Products, barcodes, ingredients, allergens, nutrition	Open Food Facts (filtered to Canada + India + common)	Barcode autofill, allergen labels	ODbL: credit OFF; share improvements to their data	200-600 MB
Nutrient composition	Health Canada Canadian Nutrient File; USDA FoodData Central	Nutrition for generic/produce items	Open licence / public domain	< 50 MB
Shelf life and storage	USDA FoodKeeper; USDA Agricultural Handbook 66	Expiry defaults, storage temp, ethylene compatibility	Public domain	< 5 MB

Dataset	Source	Used for	Licence / condition	Target size
Currency rates	Bank of Canada Valet history (bundled), refreshed when internet is available	Multi-currency documents	Free with attribution	< 5 MB
Units and conversions	Our own table	kg/lb, case/unit	Ours	tiny
Tax presets	Our own table (Canadian provinces)	GST/HST/PST defaults	Ours; verified yearly	tiny

How the data is used

- **Autofill:** scanning a barcode fills product name, brand, size, ingredients, allergens and nutrition (marked "suggested" until confirmed).
- **Suggestions:** typing "basmati rice 5 kg" suggests category, HS code 1006.30, shelf life and storage area.
- **Notifications:** expiry alerts, storage temperature mismatches, ethylene conflicts (e.g. bananas stored with leafy greens), allergen warnings on labels.
- **Import enrichment:** fills blanks during import (Section 12).

Rules

- Reference data never overwrites business data; it only suggests.
- Business edits are kept separately so dataset updates don't erase them.
- Datasets update with app updates (USB bundle); each shows its version and date in Settings.
- Licence credits appear in Settings → About.
- Indian Food Composition Tables (IFCT 2017) are excluded until redistribution rights are confirmed.

14. Security and privacy

Security uses layers: the network keeps strangers out, the app checks who you are and what your role allows, and sensitive data is encrypted at rest and in backups.

Layer	Control
Network	WPA2/WPA3 staff Wi-Fi; customer guest network isolated; router admin password changed; WPS off
Device	Each device paired with a one-time code; device key revocable from the hub
Transport	HTTPS only on the LAN with the business's own local certificate
Login	Password (min 10 chars) for back-office; PIN for tills; lockout after 5 failed attempts; auto-lock after inactivity
Authorisation	Role permissions checked on the hub (never only in the browser)
Approvals	Refunds, voids, discounts and stock adjustments above limits need a manager PIN
Sensitive fields	SIN, bank details and pay rates encrypted in the database; visible to owner only
Backups	Encrypted with a key derived from the owner's recovery code
Audit	Every change in the event log; money actions highlighted in an audit report
Hub access	SSH off by default; key-only when on; admin UI limited to owner
Privacy	Collect only needed personal data; retention rules (e.g. messages 1 year, ex-employee records per BC requirements); export/delete a person's data on request (BC PIPA)
Physical	Hub in a locked cabinet or out of public reach; USB backup taken home weekly (optional rotation)

Threats specifically handled: a stolen till (revoke device), a cashier abusing refunds (limits + audit), a lost owner password (recovery code), a dead SD card (backups + spare card), a customer on the Wi-Fi (guest isolation).

15. Reliability and operations (24/7)

The target is that no single failure stops sales: tills keep selling through a hub outage, the hub restarts itself, and a dead hub is replaced from backup in about 5 minutes.

Failure modes and responses

Failure	Prevention	Recovery	Impact on sales
App crash	Tested releases	systemd restarts in seconds	None (offline queue)
System freeze	Hardware watchdog	Auto reboot < 1 min	None
Power blip / outage	UPS on hub + router	Clean shutdown on low battery; auto start	None (UPS) / tills on battery continue offline
SD card corruption	High-endurance card, WAL, low writes, read-only system partition (optional overlay)	Swap spare card, restore last backup	Tills offline until swap
Disk full	Log rotation, alerts at 80%	Cleanup task	None
Wrong clock	RTC module, time check at boot	Alert + block till open if clock invalid	Prevented
Wi-Fi drop	Wired hub; tills cache data	Auto reconnect, queue uploads	None
Router failure	Spare router config documented	Replace router, same SSID/password	Tills offline
Bad update	Backup before update; versioned migrations	Automatic rollback	None
Hub hardware dead	Nightly backups; spare card/hub	Restore on new hub ~5-15 min	Tills offline meanwhile

Backups

- Nightly at 02:00 (configurable): SQLite online backup + uploaded files to USB drive.
- Keep 14 daily + 8 weekly + 12 monthly copies.

- Each backup verified (integrity check + checksum) and logged; failure raises an urgent task.
- Manual "Backup now" before imports, updates and period closes.
- Restore wizard: pick a backup, preview counts, restore.

Health page (owner only)

Hub uptime, CPU temperature, RAM, disk free, last backup, clock source and drift, connected devices, queue backlog per device, app version, dataset versions.

Updates

- We publish a signed update bundle (app, migrations, datasets).
- Owner copies it to USB (or downloads when internet is present); hub verifies signature, backs up, applies, health-checks, and rolls back on failure.
- Updates scheduled outside trading hours.

Support routine (for the business)

- Daily: none (automatic).
- Weekly: glance at health page; optionally take the backup USB off-site.
- Monthly: test restore to the spare card (guided).

16. AI readiness (premium tier)

AI is not in v1, but v1 is built so AI plugs in later without a rewrite. The premium tier ships Chedam pre-installed on a more powerful device (e.g. mini PC with 16 GB RAM or a Pi 5 with an AI accelerator) that runs local models offline.

What v1 does now to prepare

- **Clean, consistent data model** with clear names and units — AI reads it reliably.
- **Event log** of every change — history for forecasting and explanations.
- **Task engine** — AI later adds suggestions as tasks, using the same approval flow.
- **"Suggested" vs "confirmed" fields** — already used by datasets; AI suggestions reuse it.
- **Pluggable service interface** on the hub: `suggest()`, `classify()`, `ask()` with a rules-based implementation in v1, swapped for AI in premium.
- **Text search index** over products and parties — later becomes an embeddings index.

Planned AI features (by tool, smallest that works)

Feature	Approach	Runs on
Column mapping in imports	Small embedding model + synonyms	Premium hub or back-office PC
HS code and category suggestions	Embedding similarity on HS descriptions	Premium hub
Demand forecast and reorder quantities	Statistical models (exponential smoothing), no LLM	Any hub
Receipt and invoice reading	OCR (Tesseract/PaddleOCR) + small model to extract fields	Premium hub
Ask your business ("margin on mangoes last month?")	Small local LLM generating read-only SQL, validated	Premium hub
Feedback sentiment	Small classifier	Premium hub
Drafting vendor emails, summaries	Small local LLM	Premium hub

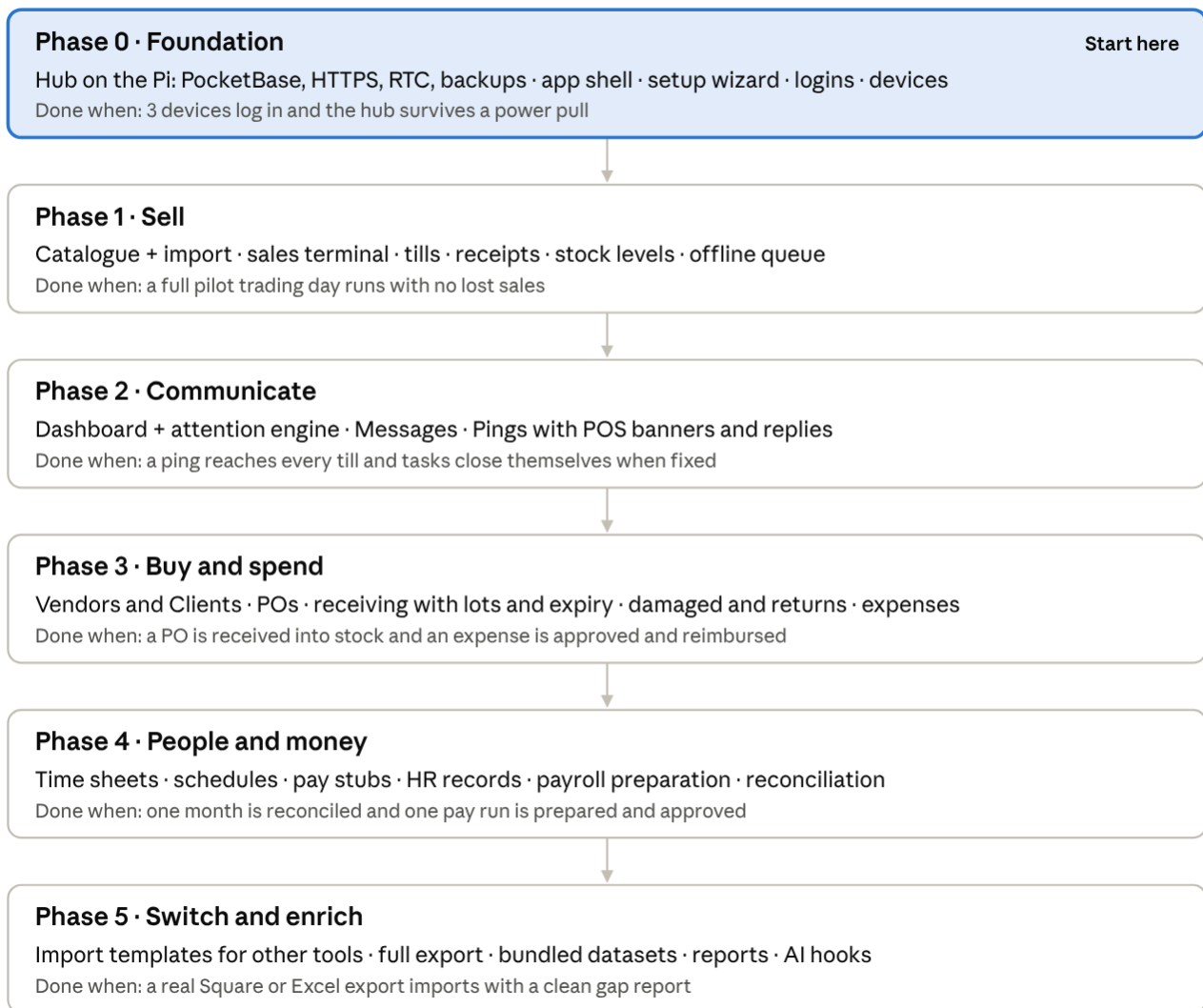
Guardrails

- AI only suggests; a person confirms before anything is saved or sent.
- AI queries are read-only; no AI path can change money records directly.
- Model runtimes: llama.cpp / Ollama / ONNX Runtime; open-weight models with commercial-friendly licences only.
- Core app never requires AI; the premium module can be removed without affecting data.

17. Delivery phases and development process

Chedam is built in six phases; each ends with a working, testable product on the Pi, so the pilot shop can use it before the next phase starts.

Six phases, each usable on its own before the next starts



delivery roadmap · 6 phases with exit criteria

Phase 0 is the foundation every module depends on; Phases 2–5 can be reordered if the pilot business needs something sooner.

Build process per phase

1. **Mini-spec (half a page):** screens, fields, rules for that phase, referencing this document.
2. **Schema first:** add or change tables as PocketBase migrations; sample data generator updated.
3. **Build:** hub hooks and client screens.
4. **Test on the Pi:** with sample data and at least 3 devices; pull the power once mid-sale.
5. **Pilot:** run in the shop; collect issues as tasks.
6. **Fix and tag a release:** update bundle created; decisions log updated.

Repository layout

```
chedam/
  hub/          # pb_hooks, pb_migrations, scripts (backup, print, health)
  client/       # Svelte PWA source
  datasets/     # build scripts + packed SQLite files
  docs/         # mini-specs, conventions, decisions
  tools/        # sample-data generator, update-bundle builder
  CONVENTIONS.md # naming, colours, folder rules for every session
```

Development tools (all free)

VS Code, Git + private GitHub repo, Node.js + Vite (build on the PC, never on the Pi), PocketBase (runs on Windows too for local development), Raspberry Pi Imager, SSH, DB Browser for SQLite, DuckDB CLI.

Working with Claude efficiently (fewer tokens)

- One phase, one module per session; start by pointing to this spec and `CONVENTIONS.md`.
- Work directly in the Chedam folder so files are edited in place, not pasted.
- Ask for edits to specific files, not full rewrites.
- Keep a small sample-data set for tests.
- Batch questions into one message; record decisions in Section 18 so they are never re-discussed.
- Reuse components (buttons, tables, forms, charts) from a shared UI kit built in Phase 0.

Definition of done (every module)

- ☐ Works offline where the spec says it should
- ☐ Permissions enforced on the hub
- ☐ Every write appears in the event log
- ☐ Relevant attention rules create and close tasks
- ☐ Exportable to CSV/Excel
- ☐ Tested on the Pi with 3 devices and a power pull

18. Risks, open questions and decisions

Risks

Risk	Likelihood	Impact	Mitigation
Pi Zero 2 W too slow under real load	Medium	Medium	Load test with 5 devices in Phase 0; production on Pi 4/5
SD card failure in 24/7 use	Medium	High	High-endurance card, backups, spare card, SSD in production
Payroll calculation errors	Medium	High	Manual deductions in v1; verify against CRA PDOC before automating
Tax rule changes	Medium	Medium	Rates in settings, not code; yearly review
Browser limits (HTTPS cert trust on phones)	Medium	Medium	Guided certificate install in device pairing; test iOS + Android early
Dataset licence obligations (ODbL)	Low	Medium	Attribution page; keep OFF-derived data separable
Scope too large for one builder	High	High	Strict phases; each phase usable on its own
Offline conflicts (oversold stock)	Medium	Low	Accept and flag as tasks

Open questions

- ☐ Which devices will the pilot tills and back office use (Windows PC, Android tablet, iPad)?
- ☐ Which receipt printer model for the pilot?
- ☐ First pilot business: Amramphalam or another shop?
- ☐ Product name and licence model: free core + paid premium hardware/AI, or one-time licence?
- ☐ Which existing tools do target businesses use most (decides first import templates)?
- ☐ Languages needed beyond English (e.g. Hindi, Telugu, Punjabi)?
- ☐ Is French required for any target customers?

Decisions log

Date	Decision
2026-10-01	Offline LAN architecture with a hub device (Option 2); no internet or subscriptions required
2026-10-01	Pilot hub: Raspberry Pi Zero 2 W; production hub: Pi 4/5 or mini PC
2026-10-01	Hub server: PocketBase + SQLite; client: PWA
2026-10-01	DuckDB used for import, profiling and analytics in the back-office browser
2026-10-01	Reference datasets bundled with the app; no user downloads
2026-10-01	One app for many businesses via setup wizard and configuration
2026-10-01	AI is a later premium tier; v1 builds the foundations